4-Bit Array Multiplier from RTL to GDSII

October 25, 2026

By

Jayakanth M V (2025122004)

Tushar Gupta (2025122001)

Introduction

Design Theory

The design is a 4-bit by 4-bit unsigned array multiplier that produces an 8-bit product. Multiplication is done by shift-and-add. Every bit b_i of the multiplier B gives one partial product A AND b_i, shifted left by i places, and the four partial products are added to give the product. Row 0 of the array is only AND gates and its lowest bit is the final bit $p[0]$. Rows 1, 2 and 3 are each a chain of four full adders (a 4-bit ripple-carry adder) that adds the next partial product to the bits passed down from the row above. The lowest sum bit of each row is a finished product bit, and row 3 also produces $p[7:4]$.

For an N-bit multiplier the array uses N^2 AND gates and $N(N-1)$ full adders, so for $N = 4$ it uses 16 AND gates and 12 full adders. The longest path runs through the carry and sum chains and is about $2N$ full-adder delays, so the delay grows roughly linearly with N.

The inputs a and b are registered on every rising clock edge, the array is built from the registered values, and the result is registered into p. This gives exactly 16 flip-flops (4 + 4 + 8) with an asynchronous active-low reset `rst_n`. An input applied before clock edge 1 is captured at edge 1 and the product is captured at edge 2, so the latency is 2 clock cycles. One result is produced every cycle, so the throughput in results per second equals the clock frequency. Because both ends of the array are registers, all timing is measured from one flip-flop to the next.

Item	Value
Function	$p = a \times b$, unsigned
Ports	`clk`, `rst_n`, `a[3:0]`, `b[3:0]`, `p[7:0]`
Logic	16 AND gates, 12 full adders
Registers	16 flip-flops (`a_r`, `b_r`, `p`)
Latency / throughput	2 cycles / 1 result per cycle
Technology	SkyWater SKY130, `sky130_fd_sc_hd`, typical corner

Table 1: Design summary.

Tools Used: The Cadence Flow

The assignment describes an open-source flow (Yosys, OpenSTA, LibreLane/OpenROAD). We carried out the same stages with the Cadence tool chain on the SKY130 high-density library. Each Cadence tool takes the place of one open-source tool, and the inputs and outputs of each stage are the same kind of files.

Stage	Assignment tool	Tool we used
RTL simulation, Part A	Icarus Verilog	Cadence Xcelium
Gate-level simulation, Part B	Icarus Verilog	Cadence Xcelium (`xrun`)
Logic synthesis, Part B	Yosys	Cadence Genus
Static timing analysis, Part C	OpenSTA	Cadence Tempus
Place and route, Part D	LibreLane / OpenROAD	Cadence Innovus
DRC and layout viewing, Part D	Magic / KLayout	Cadence Innovus

Table 2: Mapping from the assignment tools to the Cadence flow.

How the Cadence Front-End Flow Works

A chip flow has two halves. The *front end* turns the idea into a verified gate-level netlist that meets timing on paper. The *back end* (physical design) turns that netlist into a layout. The front end has four steps, as shown in Figure 1.

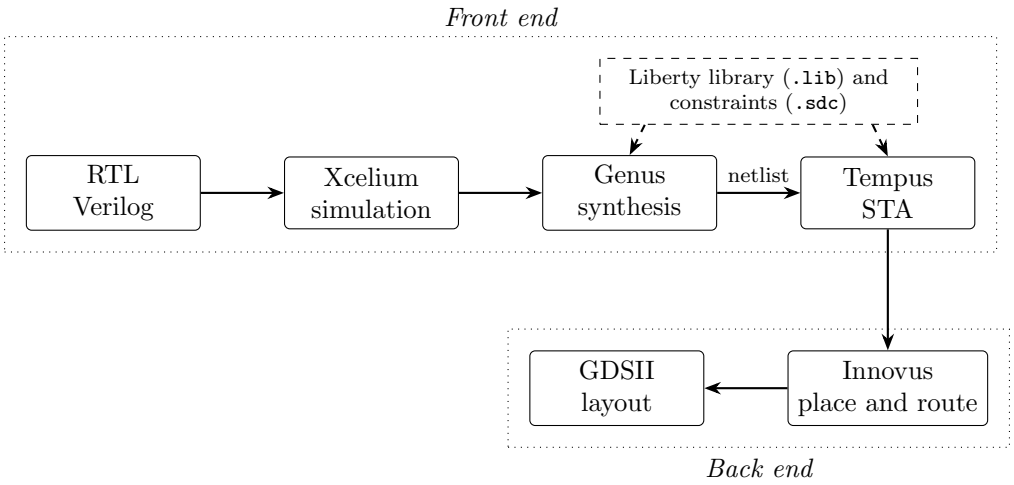


Figure 1: Cadence RTL-to-GDSII flow. The dotted boxes mark the front end and the back end.

1. RTL design and functional simulation (Xcelium)

The design is written in Verilog at register-transfer level, which describes what happens between registers on each clock edge. Xcelium compiles, elaborates and simulates the RTL together with a testbench (`xrun`). The testbench applies inputs, compares the outputs with expected values and writes a waveform. At this stage there are no delays, so the simulation only proves that the logic is functionally correct.

2. Logic synthesis (Genus)

Genus converts the RTL into a netlist of real library cells. It needs three inputs: the RTL, the Liberty file (`.lib`), which gives the area, delay and power of every standard cell, and an SDC constraint file, which gives the target clock period and the input and output delays. Synthesis runs in three commands. `syn_generic` reads the RTL and builds technology-independent generic logic. `syn_map` maps that logic onto the cells of the library. `syn_opt` then optimises the mapped netlist for timing and area, for example by resizing gates and restructuring logic. Genus writes the gate-level netlist, the SDC file for later stages, and area, gate and timing reports. The netlist is then simulated again to confirm that synthesis did not change the behaviour.

3. Static timing analysis (Tempus)

Tempus reads the netlist, the Liberty file and the constraints and checks timing without any simulation. It builds a timing graph of the design and adds up the cell delays along every path between flip-flops, from inputs to flip-flops and from flip-flops to outputs. For each path the setup slack is the time available minus the time used:

$$\text{slack} = T_{\text{clk}} - \text{path delay} - t_{\text{setup}}$$

(for input and output paths the input and output delays are added to the path delay). The worst slack (WNS) shows how much margin the design has. A positive WNS means the clock period can still be shortened. The minimum clock period is $T_{\min} = T - \text{WNS}$ and the maximum frequency is $F_{\max} = 1000/T_{\min}$ MHz, with T in ns. The path with the worst slack is the critical path.

4. Verification of the netlist

Gate-level simulation re-runs the same testbench on the synthesized netlist with the library cell models. Logic equivalence checking (Conformal) can also be used to prove formally that the netlist implements the same function as the RTL.

From the front end to the back end

The netlist from Genus and the constraints are the inputs to Innovus. Innovus performs floorplanning, power planning, placement, clock tree synthesis, routing, timing optimisation and verification, and writes the final GDSII layout. This is covered in Part D.

Part A: RTL Design and Verification

Design Files

The design is split into two modules. `full_adder.v` describes one full adder with sum $= a \oplus b \oplus c_{in}$ and $c_{out} = ab + bc_{in} + ac_{in}$. `mult_array.v` instantiates 12 of them one by one, forms the 16 partial products with AND gates, and registers `a`, `b` and the product `p` with an asynchronous active-low reset. The testbench `tb_mult.v` generates a 10 ns clock, holds `rst_n` low for the first two cycles, applies all 256 combinations of a and b, waits for the 2-cycle latency, compares `p` with $a \times b$, prints a line for every mismatch, prints one summary line, and writes `mult.vcd`. The RTL was simulated with Cadence Xcelium on EDA Playground.

Block Diagram

Figure 2 shows the multiplier. Each row has four AND gates for the partial product $A \cdot b_i$. Row 0 only has AND gates and gives P_0. Rows 1 to 3 each have four full adders whose carries ripple to the left. The first full adder of every row and the unused inputs take a constant 0. The input registers on A and B and the output register on P are not drawn.

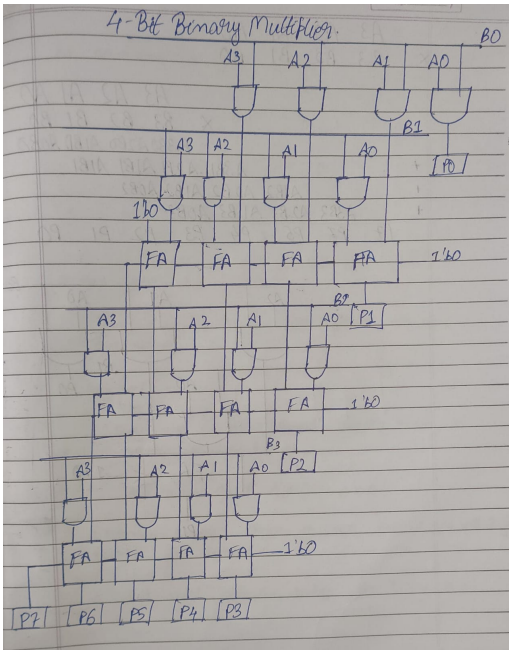


Figure 2: Hand-drawn block diagram of the 4-bit array multiplier: 16 AND gates and 12 full adders.

Verification Result

With all 256 input combinations applied, the testbench prints the summary line below.

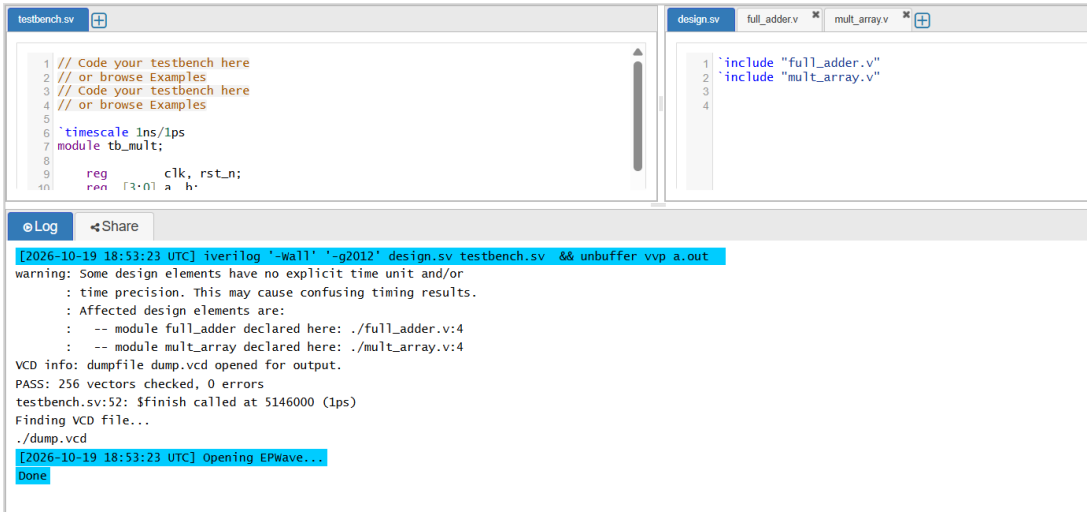


Figure 3: Testbench output: `PASS: 256 vectors checked, 0 errors`.

Hand-Worked Example

The example operands are chosen from the last digit of Tushar Gupta's roll number (2025122001): $A = 1 + 5 = 6$ and $B = 7$.

```
          0 1 1 0      A = 6
        × 0 1 1 1      B = 7
  ---------------------------------------------------
          0 1 1 0      pp0 = A AND b0, b0 = 1
        0 1 1 0        pp1 = A AND b1, b1 = 1, shifted 1
      0 1 1 0          pp2 = A AND b2, b2 = 1, shifted 2
    0 0 0 0            pp3 = A AND b3, b3 = 0, shifted 3
  ---------------------------------------------------
  0 0 1 0 1 0 1 0      P = 42
```

The check is $6 + 12 + 24 + 0 = 42 = 00101010_2$. The same result follows from the array equations, row by row:

Row	Adds	Result	Finished bit / passed down
0	$pp_0 = 0110$	—	$p[0] = 0$; $r_0 = 0011$
1	$pp_1 + r_0 = 0110 + 0011$	1001	$p[1] = 1$; $r_1 = 0100$
2	$pp_2 + r_1 = 0110 + 0100$	1010	$p[2] = 0$; $r_2 = 0101$
3	$pp_3 + r_2 = 0000 + 0101$	0101	$p[3] = 1$; $p[7:4] = 0010$

Table 3: Row-by-row evaluation for 6×7: $p = 0010\,1010_2 = 42$.

Waveform

Figure 4 shows the same multiplication in the simulation. The input $b = 7$ is applied while $a = 6$, it is captured by the input registers at the next rising edge, and the product $p = 42$ appears at the output register at the second rising edge. This is the 2-cycle latency.

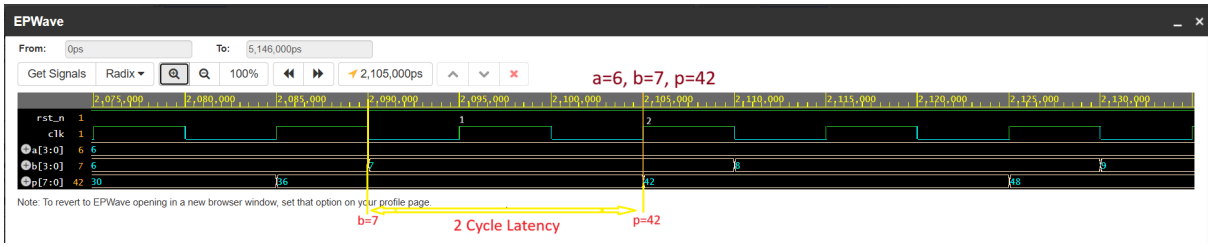


Figure 4: Waveform of `clk`, `rst_n`, `a`, `b` and `p` for $6 \times 7 = 42$, with the 2-cycle latency marked.

Checking That the Testbench Catches Errors

To confirm that the testbench detects faults, the sum equation of the full adder was changed temporarily to `sum = a ^ b`, which drops the carry input. The full adder was tested with its own small testbench, and then restored.



Figure 5: With `sum = a ^ b` the testbench prints `FAIL: 8 vectors checked, 4 errors`. The four mismatches are the input combinations with $c_{in} = 1$.

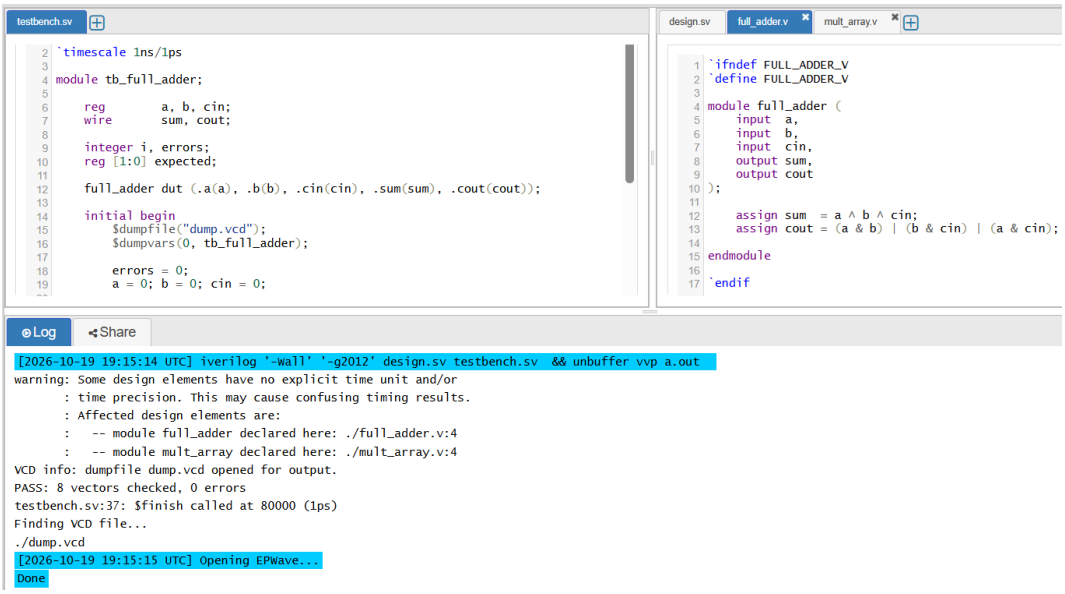


Figure 6: After undoing the change (`sum = a ^ b ^ cin`) the same testbench prints `PASS: 8 vectors checked, 0 errors`.

Part B: Logic Synthesis

Synthesis Script

Synthesis was run in Genus with a Tcl script, `synth/synth.tcl`. It reads the typical-corner library `sky130_fd_sc_hd__tt_025C_1v80`, reads and elaborates the RTL, applies the constraints, runs the three synthesis commands and writes the results. The clock period was set to 10 ns with 1.0 ns input and output delays, so that the design is not timing-limited during synthesis and the same constraints can be reused in Part C.

Step	Genus command	Purpose
1	`set_db library`	Select the Liberty cell library
2	`read_hdl`, `elaborate`	Read `full_adder.v` and `mult_array.v` and build the design
3	`create_clock`	Clock `clk` with a period of 10 ns
4	`set_input_delay`, `set_output_delay`	1.0 ns on all inputs and outputs
5	`ungroup`	Flatten the hierarchy so that the full adders are optimised together
6	`syn_generic`	Convert the RTL into generic logic
7	`syn_map`	Map the generic logic to library cells
8	`syn_opt`	Optimise the mapped netlist for timing and area
9	`report_area`, `report_gates`, `report_timing`	Write the area, cell and timing reports
10	`write_hdl`, `write_sdc`	Write the gate-level netlist and the constraints

Table 4: Steps in `synth.tcl`.

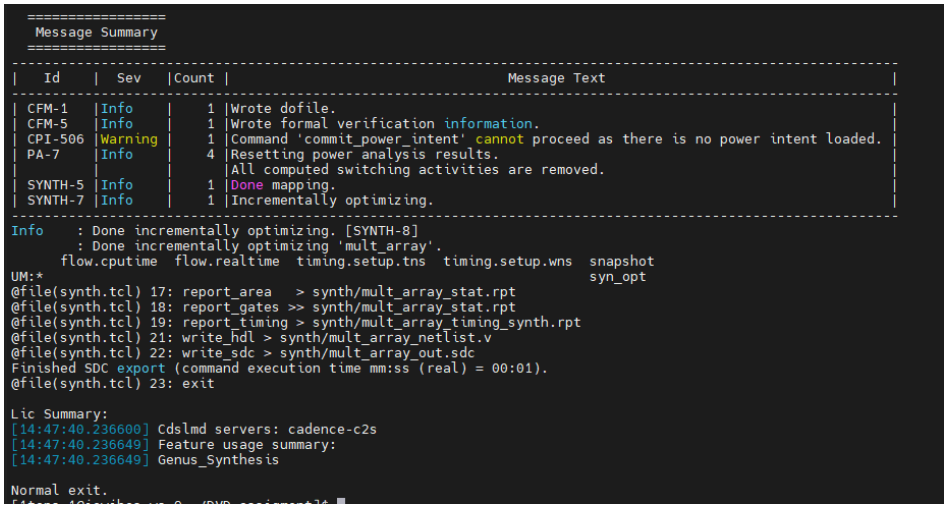


Figure 7: End of the Genus run: mapping and incremental optimisation finish, the reports, netlist and SDC are written, and Genus exits normally.

Results

The results below are taken from `synth/mult_array_stat.rpt`. The flip-flop area is the number of flip-flops times the area of one `dfrtp_1` cell: $16 \times 25.024 = 400.384\ \mu\text{m}^2$.

Metric	Value
Total number of cells	62
Number of flip-flops	16
Total cell area (μm^2)	711.933
Flip-flop area (μm^2)	400.384
Flip-flop area (% of total)	56.2
3 most-used cell types and their counts	`dfrtp_1` (16), `nand2_1` (14), `o2bb2ai_1` (8)
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

Table 5: Part B results.

The 62 cells are 16 sequential cells (400.384 μm^2, 56.2 %) and 46 logic cells (311.549 μm^2, 43.8 %). There are no physical cells at this stage. The full list of cells is given in Table 6.

Cell	Count	Area (μm^2)
sky130_fd_sc_hd__dfrtp_1	16	400.384
sky130_fd_sc_hd__nand2_1	14	52.550
sky130_fd_sc_hd__o2bb2ai_1	8	70.067
sky130_fd_sc_hd__nor2_1	4	15.014
sky130_fd_sc_hd__xor2_1	4	35.034
sky130_fd_sc_hd__a21oi_1	3	15.014
sky130_fd_sc_hd__a2bb2oi_1	3	26.275
sky130_fd_sc_hd__and2_1	3	18.768
sky130_fd_sc_hd__fa_1	2	40.038
sky130_fd_sc_hd__maj3_1	1	10.010
sky130_fd_sc_hd__nand2b_1	1	6.256
sky130_fd_sc_hd__nor2b_1	1	6.256
sky130_fd_sc_hd__o21bai_1	1	7.507
sky130_fd_sc_hd__xnor2_1	1	8.758
Total	62	711.933

Table 6: All cells in the synthesized netlist.

Netlist Simulation

To show that synthesis did not change the behaviour, the same testbench `tb_mult` was run in Xcelium on the synthesized netlist `synth/mult_array_netlist.v` together with Verilog models of the library cells. All 256 input combinations were applied. The testbench printed `PASS: 256 vectors checked, 0 errors`, and the simulation finished at 5146 ns, the same end time as the RTL simulation. Figure 8 shows the SimVision waveform window and the console.

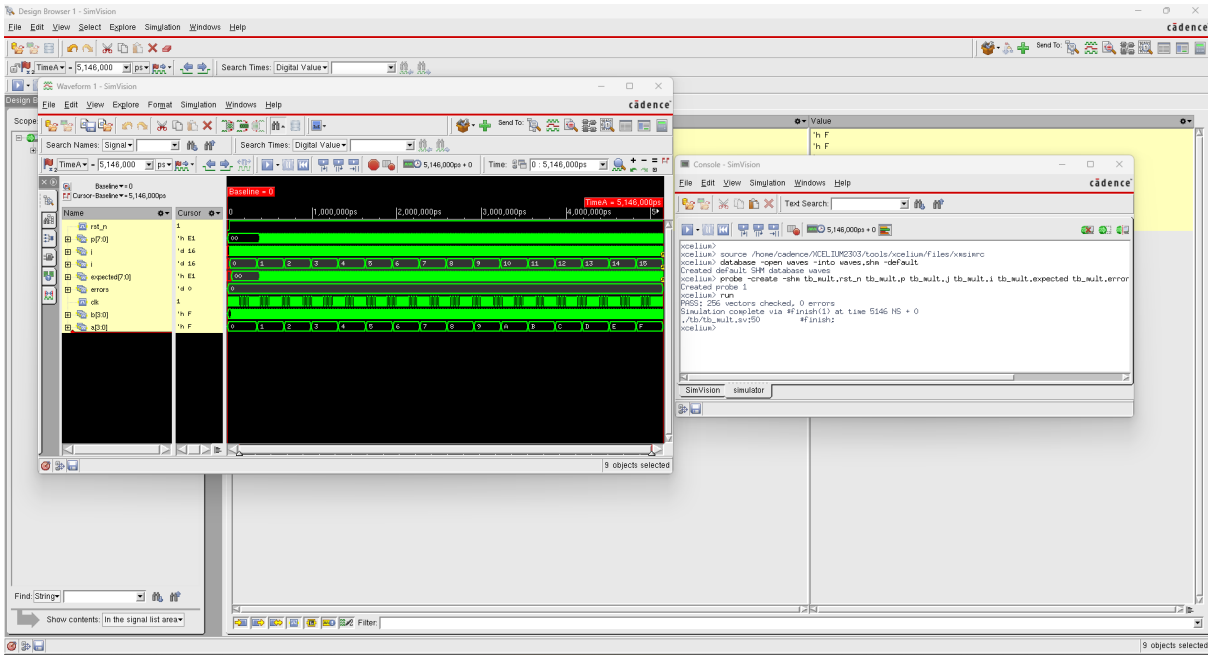


Figure 8: Netlist simulation in Xcelium: `PASS: 256 vectors checked, 0 errors`.

Part C: Static Timing Analysis

STA Script

Timing was analysed in Tempus on the synthesized netlist with the typical corner (`tt_025C_1v80`) only. The script `sta/sta.tcl` sets the clock period T in a variable, reads the library and the netlist, sets `mult_array` as the top module, defines the clock `clk` with period T, applies a 1.0 ns input delay to all inputs except the clock and a 1.0 ns output delay to all outputs, and writes the worst setup path to `sta/mult_array_timing.rpt` with `report_timing`. The clock is ideal, so there is no clock network delay or skew at this stage.

Finding Tmin and Fmax

The script was run with $T = 10$ ns. The worst path met timing with a slack of 7.116 ns. Since the slack is the margin left in the clock period,

$$T_{\min} = T - \mathrm{WNS} = 10 - 7.116 = 2.884 \text{ ns}, \qquad F_{\max} = \frac{1000}{T_{\min}} = \frac{1000}{2.884} \approx 346.7 \text{ MHz}.$$

The path report agrees with this: the data arrives 2.759 ns after the clock edge and the flip-flop setup time is 0.125 ns, so the shortest period that works is $2.759 + 0.125 = 2.884$ ns.

Critical Path

The critical path starts at the clock pin of `b_r_reg[1]` and ends at the data pin of `p_reg[6]`. It goes from the input register to the output register through the multiplier array. Table 7 lists the cells in order.

No.	Instance	Cell	Delay (ns)	Arrival (ns)
1	`b_r_reg[1]`	`dfrtp_1` (clock to Q)	0.353	0.353
2	`g1342__4319`	`nand2_1`	0.131	0.484
3	`g1332__5107`	`nor2_1`	0.214	0.698
4	`g1326__1666`	`nor2_1`	0.163	0.861
5	`g1322__9945`	`o2bb2ai_1`	0.182	1.044
6	`g1315__1881`	`fa_1` (B to COUT)	0.435	1.478
7	`g1312__5122`	`fa_1` (B to SUM)	0.567	2.046
8	`g1306__1617`	`a2bb2oi_1`	0.130	2.176
9	`g1301__4319`	`maj3_1`	0.405	2.581
10	`g1298__5107`	`o2bb2ai_1`	0.169	2.750
11	`p_reg[6]`	`dfrtp_1` (D pin)	0.009	2.759

Table 7: Critical path from `sta/mult_array_timing.rpt` at $T = 10$ ns. Required time $= 10 - 0.125 = 9.875$ ns, slack $= 9.875 - 2.759 = 7.116$ ns.

There are 11 cells on the path: the launching flip-flop, 9 combinational cells and the capturing flip-flop. Of the 9 combinational cells, two are `fa_1` full-adder cells and one is a `maj3_1` cell, which computes the carry (majority) function of a full adder. The other six (`nand2_1`, two `nor2_1`, two `o2bb2ai_1` and `a2bb2oi_1`) are simple gates.

The estimate in Section 2 of the assignment is about $2N = 8$ full-adder delays on the longest path. Our path has only 2 full-adder cells plus the carry cell, which together take $0.435 + 0.567 + 0.405 = 1.407$ ns, about half of the arrival time. The reason is that the design was flattened before synthesis, so Genus did not keep the 12 full-adder modules as 12 `fa_1` cells. Most of them were rebuilt from smaller gates, which are faster (0.13 to 0.21 ns each) than an

`fa_1` (0.43 to 0.57 ns). If all 8 stages were `fa_1` cells, the path would take roughly $8 \times 0.5 = 4$ ns in the array alone, but the whole combinational path here takes $2.750 - 0.353 = 2.397$ ns. The estimate of $2N$ stages is therefore right about how many adder stages the signal crosses, but the delay per stage is smaller after optimisation.

Results

Metric	Value
Worst slack at $T = 10$ ns (ns)	7.116
$T_{\min}$ (ns)	2.884
$F_{\max}$ (MHz)	346.7
Slack at $T = T_{\min}$ (ns)	-0.004 (at $T = 2.88$ ns)
Critical path start point	`b_r_reg[1]/CLK`
Critical path end point	`p_reg[6]/D`
Number of cells on the critical path	11 (9 combinational + 2 flip-flops)
Latency $= 2 \times T_{\min}$ (ns)	5.768
Throughput (million results per second)	346.7

Table 8: Part C results.

The latency is two clock cycles, $2 \times 2.884 = 5.768$ ns. One result is produced every cycle, so the throughput in million results per second equals $F_{\max}$.

To confirm $T_{\min}$, the script was run again with $T = 2.88$ ns, which is $T_{\min}$ rounded to 0.01 ns. The critical path is the same as before (arrival 2.759 ns, setup 0.125 ns, required time 2.755 ns) and the slack is -0.004 ns, which is close to zero as expected. The small negative value appears only because 2.884 ns was rounded down to 2.88 ns. At $T = 2.89$ ns the slack would be positive.

Part D: Physical Design

Flow and Settings

Place and route was done in Innovus with a Tcl script, `pnr/pnr.tcl`, on the SKY130 high-density library. The files that replace the LibreLane `config.json` are the script itself, the multi-mode multi-corner file `pnr/mmmc.tcl` (typical-corner library and the constraints) and the SDC file written by Genus. Innovus needs the technology LEF and the cell LEF, which describe the metal layers and the cell shapes, in addition to the netlist.

Step	Stage	What the script does
1	Floorplan	Fixed 150 × 150 μm die, as required for such a small design, with standard-cell rows (site `unithd`) inside a core margin of about 5.5 μm, and the input and output pins placed on the left and right edges
2	Power grid	Well-tap cells every 28 μm, a power ring on `met4` and `met5`, `met4` stripes, and standard-cell power rails joined to the grid with `sroute`
3	Placement	`place_design`, then `optDesign -preCTS`
4	Clock tree synthesis	`ccopt_design` with the library clock buffers and inverters, then `optDesign -postCTS` for setup and hold
5	Routing	`routeDesign` on `met1` to `met5`
6	Signoff	Parasitic extraction, `timeDesign -postRoute` for setup and hold, `verify_drc`, `verifyConnectivity`
7	GDSII	Filler cells added, then `streamOut` merges the standard-cell layouts into `pnr/mult_array.gds`

Table 9: Place-and-route stages in `pnr.tcl`.

The clock period is $1.2 \times T_{\min}$ from Part C, rounded up to the next 0.5 ns: $1.2 \times 2.884 = 3.46$ ns, which gives **CLOCK_PERIOD = 3.5 ns**. The SDC file uses this period with the same 1.0 ns input and output delays as before.

Running the Flow

The flow is run from the project folder with `innovus -init pnr/pnr.tcl`. It must finish with no DRC errors and a setup slack of at least zero, and if timing fails the clock period is to be increased by 0.5 ns and the flow run again. Table 10 records the timing attempts.

Attempt	Period (ns)	Setup slack (ns)	Hold slack (ns)	DRC errors	Result
1	3.5	0.676	0.566	0	Accepted

Table 10: Timing attempts of the place-and-route flow.

The first attempt at 3.5 ns met timing with a positive setup slack of 0.676 ns and a positive hold slack of 0.566 ns, and the DRC check reported no violations, so the period did not have to be increased and only one timing attempt was needed. The flow was also executed a few times while the scripts were being set up, to fix tool settings such as the technology LEF, the clock waveform in the SDC file and the pin placement. None of these runs failed because of timing, so they are not counted as attempts.

Results

The numbers were read from the Innovus reports instead of a `metrics.json` file: `summary.rpt` for the cell counts, area and wirelength, `timing_setup/mult_array_postRoute.summary` for the setup slack, `drc.rpt` for the DRC result, and `power.rpt` for the power.

Metric	Value
CLOCK_PERIOD used (ns)	3.5
Number of runs needed	1 (timing was met at the first period tried)
Core area (μm^2)	19318.215 (die 150×150 μm)
Cells after synthesis (Part B)	62
Cells after place and route	2696
Kinds of cells added	306 well-tap cells and 2328 filler cells (`fill_1`: 329, `fill_2`: 281, `fill_4`: 27, `fill_8`: 1691); no buffers or decap cells
Wirelength (μm)	1922.68
Setup slack (ns)	0.676
Post-route $F_{\max}$ (MHz)	354.1
$F_{\max}$ from Part C (MHz)	346.7
DRC errors	0
LVS errors	0 (connectivity check)

Table 11: Part D results.

LVS and Connectivity Check

Innovus does not have an LVS command, so the layout was checked against the netlist with the Innovus `verifyConnectivity` command, which compares the routed wires with the connections of the netlist. It reported no opens and no shorts on the signal nets, and no unconnected pins on the 62 logic cells. The only entries in `connectivity.rpt` are 9 dangling `met4` wire ends, which are the ends of the power stripes and not signal nets. Together with the DRC result of 0 violations, this gives 0 errors in the LVS row of Table 11.

Layout

Figure 9 shows the whole routed die in the Innovus viewer. The die is 150×150 μm. The input pins (`clk`, `rst_n`, `a`, `b`) are on the left edge and the eight product pins `p[7:0]` are on the right edge. The power ring runs around the core, with vertical `met4` power stripes every 30 μm and horizontal power rails along every cell row. The logic is a small cluster near the input pins at the left, and the rest of the core is filled with tap and filler cells.

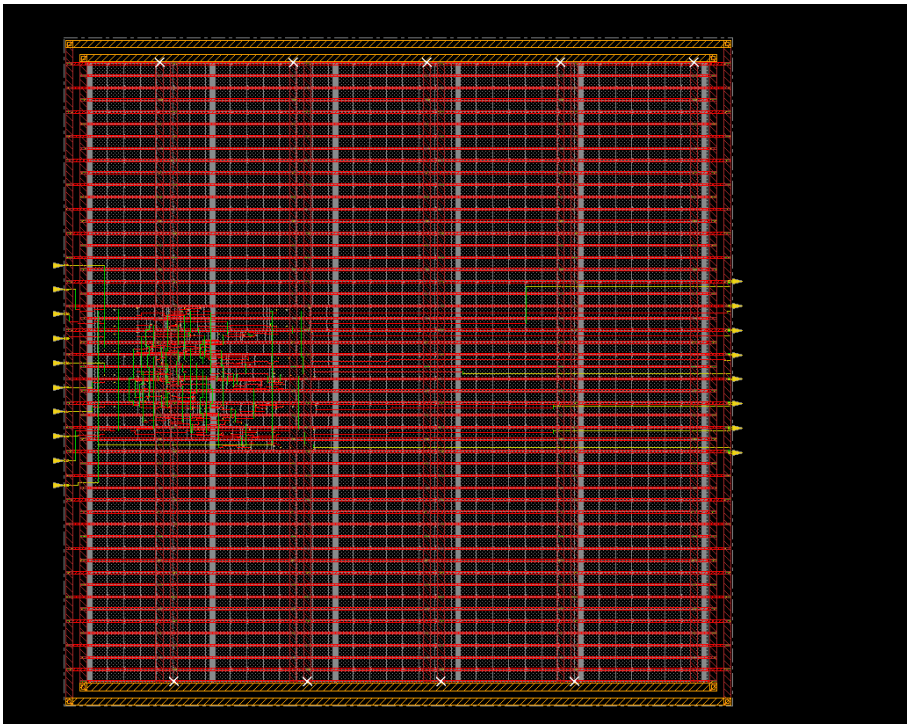


Figure 9: Whole die of the routed design.

Figure 10 is a zoomed view of the logic area. The horizontal bands are the standard-cell rows, which are filled with the logic cells and the filler cells (the instance names `FILLER_T_1_*` are visible). The `VPWR` and `VGND` rails run along the rows. The signal wires between the cells are the horizontal `met1` wires (red) and the vertical `met2` wires (green), joined by vias.

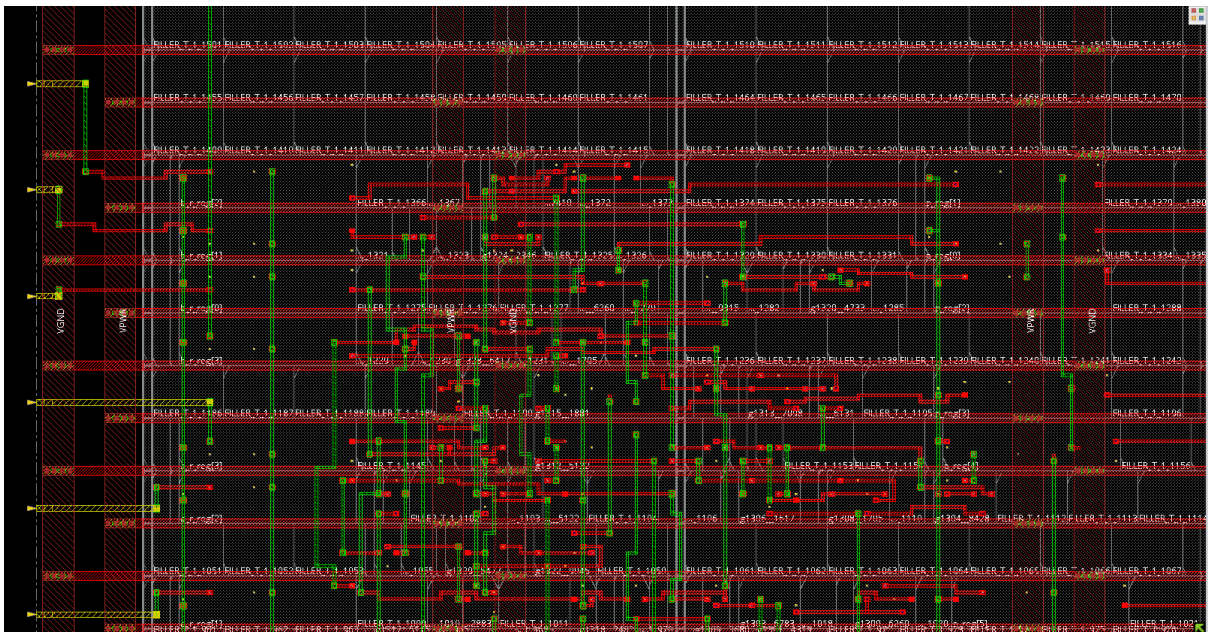


Figure 10: Zoomed view of the standard-cell rows and the metal wires.

Discussion

Post-route Fmax. With the setup slack of 0.676 ns at a 3.5 ns clock, the post-route maximum frequency is $1000/(3.5 - 0.676) = 1000/2.824 \approx 354.1$ MHz. This is about 2 % higher than the

346.7 MHz found in Part C, not lower. The logic cells in the routed design are exactly the same as in the synthesized netlist (Table 6), so no gates were resized. The difference therefore comes only from the timing set-up: Part C used an ideal clock and no wires, while the post-route analysis uses extracted wire parasitics and the real clock net. Because the design is only about 140 μm wide, the wires add very little delay, and the two results are close.

Cell count. The design grew from 62 cells after synthesis to 2696 cells after place and route, but the 62 logic and flip-flop cells are unchanged. The 2634 added cells are all physical cells that carry no logic. The 306 `tapvpwrvgnd_1` cells tie the n-well and the substrate to the supply so that latch-up cannot occur, and they were placed every 28 μm along the rows. The 2328 filler cells (`fill_1`, `fill_2`, `fill_4`, `fill_8`) close the gaps in the rows so that the wells and power rails are continuous and the density rules are met. No clock buffers or decap cells appear in the final cell list: the clock net drives only 16 flip-flops, so clock tree synthesis did not need to add any.

Utilisation. The core is 19318 μm^2, but the logic cells take only 711.9 μm^2, a density of 3.7 % before the filler cells are added. This is because the die size was fixed at 150×150 μm by the assignment, which is much larger than this design needs.

Timing, power and checks. The final setup check has no violating paths (worst slack 0.676 ns), and the hold check also passes with a worst hold slack of 0.566 ns. There are no maximum-capacitance, transition or fanout violations. The total power is 0.298 mW, of which 80 % is internal power, mostly in the flip-flops, with a very small leakage component. Routing finished with no DRC or antenna violations, and `verify_drc` reports no DRC violations. The connectivity check reports no opens or shorts on the signal nets. Its only messages are 9 dangling wire ends on `met4`, which are the ends of the power stripes that extend past the last cell row. They carry no signal and do not affect the design. The well-bias pins VPB and VNB of the cells are not routed as signals, because the tap cells tie the wells to the supply.

Questions for the Report

Q1

How many cells did synthesis produce for the logic, and why is the number not simply 16 + 12?

Synthesis produced $62 - 16 = 46$ cells for the logic, which is more than $16 + 12 = 28$. The reason is that a full adder is not one cell in the library, and only two `fa_1` cells and one `maj3_1` cell were used. After the hierarchy was flattened, Genus rebuilt most adders from simple gates (14 `nand2_1`, 4 `nor2_1`, 4 `xor2_1`, 8 `o2bb2ai_1`, 3 `a2bb2oi_1`, 3 `a21oi_1` and others), because these gates are smaller and faster than an `fa_1` on the critical path. The AND gates of the partial products were also merged into inverting gates such as NAND, NOR and `o2bb2ai`, so there are only 3 `and2_1` cells instead of 16. In addition, four of the 12 adders (`fa1`, `fa3`, `fa6` and `fa7` in our RTL) have a constant 0 input, so they reduce to half adders, which need fewer cells.

Q2

What is the critical path, and how does it compare with 2N full-adder delays?

The critical path runs from the input register `b_r_reg[1]` to the output register `p_reg[6]`. It has 11 cells: the two flip-flops and 9 combinational cells (`nand2_1`, two `nor2_1`, two `o2bb2ai_1`, two `fa_1`, `a2bb2oi_1` and `maj3_1`). The estimate of $2N = 8$ full-adder delays is therefore close

to the 9 logic levels we found, but the delay per level is much smaller: the combinational path takes 2.397 ns, about 0.30 ns per level, while a single `fa_1` takes 0.43 to 0.57 ns. Only three of the 9 cells (two `fa_1` and one `maj3_1`, 1.407 ns together) are adder cells, because synthesis replaced the other adder stages with faster simple gates.

Q3

What percentage of the total cell area is flip-flops, and why is it large?

The 16 flip-flops take 400.384 μm^2 out of 711.933 μm^2, which is 56.2 % of the cell area. The percentage is large for two reasons. First, the number of flip-flops is fixed by the interface (4 + 4 input bits and 8 product bits) and does not shrink with the size of the logic, while the logic is tiny, with only 46 cells of about 6.8 μm^2 each on average. Second, a flip-flop with asynchronous reset (`dfrtp_1`, 25.024 μm^2) is large compared with a gate: it is about 6.7 times the area of a `nand2_1` (3.754 μm^2) and about 2.9 times that of an `xor2_1` (8.758 μm^2).

Q4

Why is the post-route Fmax lower, and which cells did the flow add?

In our results the post-route $F_{\max}$ is not lower: it is 354.1 MHz, against 346.7 MHz from Part C, a difference of about 2 %. It is normally lower because after routing the real wire resistance and capacitance add delay and the clock tree adds skew. Here the design is so small, with a total wire length of only 1922.68 μm, that the wires add almost no delay, and Part C used an ideal clock and a different timing engine, so the two values stay close. The flow added 306 tap cells and 2328 filler cells (`fill_1`: 329, `fill_2`: 281, `fill_4`: 27, `fill_8`: 1691), taking the design from 62 to 2696 cells. The tap cells (`tapvpwrvgnd_1`) connect the n-well and the substrate to the supply rails so that latch-up cannot occur. The filler cells fill the gaps in the rows so that the wells and power rails are continuous and the density rules are met. No clock buffers were added, because the clock net drives only 16 flip-flops, and no decap cells were used.

Q5

What do DRC and LVS check, and why must both pass?

A DRC (design rule check) verifies that the layout geometry obeys the foundry's manufacturing rules, such as the minimum width and spacing of each layer, the enclosure of vias, density and antenna rules. A layout that breaks these rules cannot be made reliably or may give poor yield. An LVS (layout versus schematic) check extracts the transistors and connections from the layout and compares them with the netlist, to show that every device is present and connected as intended, with no shorts, opens or missing cells. Both must pass because they check different things: a layout can be DRC-clean but implement the wrong circuit, and it can match the netlist but still be impossible to manufacture. In our flow the Innovus `verify_drc` check reported no DRC violations, and `verifyConnectivity` reported no opens or shorts on the signal nets.

Q6

How would the number of full adders and the critical path delay change for an 8-bit multiplier?

From Section 2, an N-bit array multiplier needs N^2 AND gates and $N(N-1)$ full adders, and its longest path is about $2N$ full-adder delays. For $N = 4$ this is 16 AND gates, $4 \times 3 = 12$ full adders and 8 delays. For $N = 8$ it is $8^2 = 64$ AND gates, $8 \times 7 = 56$ full adders (4.7 times as many as 12) and $2 \times 8 = 16$ delays, so the critical path delay doubles. With the delay per level measured in our design, $2.397/8 \approx 0.30$ ns, the 8-bit combinational path would take about $16 \times 0.30 = 4.8$ ns. Adding the clock-to-Q delay of 0.353 ns and the setup time of 0.125 ns

gives $T_{\min} \approx 5.27$ ns and $F_{\max} \approx 190$ MHz, compared with 347 MHz for 4 bits. The number of flip-flops would also double, from $4N = 16$ to 32.

Contributions

Jayakanth M V	Synthesis, Physical Design
Tushar Gupta	RTL, STA

Repository

The design files, scripts and reports are available on GitHub:
`https://github.com/jayash9658/4-bit-unsigned-array-multiplier/tree/main`